

Day 12: LangChain Fundamentals

5-line beginner summary

LangChain is a framework that helps developers build LLM-powered applications faster. Instead of manually writing all glue code for prompts, models, tools, memory, retrieval, and output formatting, LangChain gives reusable building blocks. It is commonly used for chatbots, RAG apps, agents, document Q&A systems, and enterprise GenAI workflows. LangChain does not replace LLMs, vector databases, APIs, or data pipelines; it connects them together. For enterprise AI, LangChain is useful for prototyping and orchestration, but production systems still need testing, monitoring, security, evaluation, and governance.

Descriptive notes

1. What LangChain is

LangChain is an open-source framework for building applications powered by LLMs. In simple words, it helps you connect an LLM with prompts, documents, vector databases, APIs, tools, memory, and business logic. LangChain's current docs describe it as a framework for building agents and LLM-powered applications, with integrations for model providers like OpenAI, Anthropic, Google, and others. ([Docs by LangChain](#))

Think of LangChain like a **GenAI application wiring framework**.

Without LangChain, you may manually write code like this:

```
User question
→ create prompt
→ call LLM API
→ parse response
→ search documents
→ add context
→ call LLM again
→ format final answer
```

With LangChain, these steps can be organized as reusable components.

2. Why frameworks are used in GenAI apps

In real GenAI apps, calling an LLM is only one part of the system. A production app may need:

Need	Example
Prompt management	Convert user input into a structured prompt
Model abstraction	Switch between OpenAI, Anthropic, IBM watsonx, Google, etc.
Retrieval	Search documents from vector DB

Need	Example
Tools	Call calculator, database, API, web search, ticketing system
Memory	Remember previous conversation context
Parsing	Return JSON, table, decision, score, or structured output
Observability	Track what prompt, context, and model output were used
Evaluation	Check hallucination, groundedness, relevance

LangChain provides standard abstractions and many integrations so developers do not need to build everything from scratch. LangChain's integration docs mention a large ecosystem across chat models, embedding models, tools, document loaders, vector stores, and more. ([Docs by LangChain](#))

3. LLM wrapper

An **LLM wrapper** is a standard way to call different LLM providers using a similar interface.

For example, without a wrapper, each provider may have different code:

```
OpenAI API style
Anthropic API style
Google Gemini API style
IBM watsonx API style
```

LangChain gives model integrations so your application can call models in a more consistent way. Current LangChain docs describe support for integrations with multiple model providers. ([Docs by LangChain](#))

Easy example

```
User: Summarize this policy.

LangChain model wrapper:
  send prompt to selected LLM
  receive response
  return answer to app
```

Why useful?

You can design your app like this:

```
app → LangChain model interface → actual LLM provider
```

So later, if your enterprise switches from one model to another, your application code may need fewer changes.

4. Prompt templates

A **prompt template** is a reusable prompt with placeholders.

Instead of writing a new prompt every time:

```
Explain leave policy for Ravi.  
Explain leave policy for Priya.  
Explain leave policy for Amit.
```

You create a template:

```
Explain the following policy question in simple language:  
  
Question: {question}  
  
Use only this context:  
{context}
```

Then LangChain fills the placeholders.

The LangChain tracing/run format describes a prompt as a template that formats inputs before passing them to a model. ([Docs by LangChain](#))

Easy example

```
Template:  
"You are an HR assistant. Answer this question: {user_question}"  
  
Input:  
"What is the maternity leave policy?"  
  
Final prompt:  
"You are an HR assistant. Answer this question: What is the maternity leave  
policy?"
```

5. Chains

A **chain** means connecting multiple steps together.

Example:

```
Step 1: Take user question  
Step 2: Format prompt  
Step 3: Call LLM  
Step 4: Parse output  
Step 5: Return final answer
```

In LangChain, a chain can be a sequence or composition of steps. ([Docs by LangChain](#))

Easy example

```
User question  
→ Prompt template  
→ LLM
```

```
→ Output parser
→ Final response
```

Real use

For a policy assistant:

```
Employee question
→ Retrieve policy documents
→ Add documents to prompt
→ Ask LLM
→ Return grounded answer with sources
```

That full flow can be organized as a chain.

6. Output parsers

An **output parser** converts the raw LLM response into a structured format.

LLMs usually return plain text. But applications often need:

```
{
  "answer": "Yes, reimbursement is allowed.",
  "confidence": "high",
  "source": "Travel Policy 2025"
}
```

LangChain docs describe an output parser as something that transforms raw model output into a structured format. ([Docs by LangChain](#))

Easy example

Raw LLM output:

```
The employee is eligible for reimbursement. Confidence: high.
```

Parsed output:

```
{
  "eligible": true,
  "confidence": "high"
}
```

Why useful?

Enterprise apps often need structured output for:

```
approval workflows
ticket routing
risk classification
policy compliance checks
dashboards
audit logs
```

LangChain also supports structured output patterns where agents can return predictable formats such as JSON objects, Pydantic models, or dataclasses. ([Docs by LangChain](#))

7. Retrievers

A **retriever** searches relevant information from a knowledge source.

In RAG, the retriever finds useful chunks from documents before the LLM generates the final answer.

Example:

```
User question:
"What is the laptop replacement policy?"

Retriever searches:
- HR policy
- IT asset policy
- employee handbook

Retriever returns:
Top 3 relevant chunks
```

LangChain describes retrievers as lookups that fetch relevant documents or context. ([Docs by LangChain](#))

Retriever vs LLM

Component	Job
Retriever	Finds relevant information
LLM	Uses that information to generate an answer

A retriever does not usually generate the final answer. It gives context to the LLM.

8. Tools

A **tool** is a function or external capability that an LLM-powered app can call.

Examples:

```
Search employee database
Check leave balance
Call calculator
Create Jira ticket
Query SQL database
Fetch latest policy from API
```

LangChain docs describe tools as callable functions with defined inputs and outputs that allow agents to fetch data, execute code, query databases, or take actions. ([Docs by LangChain](#))

Easy example

User asks:

```
How many leave days do I have left?
```

The LLM alone may not know this.

So the agent can call a tool:

```
Tool: get_leave_balance(employee_id)
Result: 12 days
```

Final answer:

```
You have 12 leave days remaining.
```

9. Memory basics

Memory means remembering useful information from previous interactions.

Example:

```
User: My name is Ravi and I work in Pune.
Later:
User: What leave policy applies to me?
Assistant can remember Ravi and Pune.
```

LangChain's current memory concept separates memory by recall scope. Short-term memory tracks the ongoing conversation within a thread/session, and LangGraph can manage this as part of the agent state using checkpointing. ([Docs by LangChain](#))

Simple types of memory

Memory type	Meaning	Example
Short-term memory	Current chat/session context	Previous 5 messages
Long-term memory	Stored user/project facts	User prefers concise answers
Application memory	Business state	Ticket ID, order ID, case status

Important warning

Memory should be used carefully in enterprise systems because it may contain sensitive data. You need access control, retention rules, encryption, and auditability.

10. RAG using LangChain

RAG means **Retrieval-Augmented Generation**.

LangChain can help build RAG by connecting:

```
documents
→ chunks
```

- embeddings
- vector database
- retriever
- prompt
- LLM
- answer

LangChain's RAG docs describe both a RAG agent and a two-step RAG chain. A two-step RAG chain retrieves context and then makes a single LLM call, which is useful for simple queries. A RAG agent can decide when to search using a retrieval tool. ([Docs by LangChain](#))

Easy RAG example: Internal policy assistant

User asks:

Can I claim taxi reimbursement for airport travel?

RAG flow:

1. User asks question
2. Retriever searches travel policy documents
3. Relevant chunks are found
4. Prompt is created with question + policy chunks
5. LLM answers using only the retrieved policy
6. Assistant gives answer with source reference

Final answer:

Yes, you can claim taxi reimbursement for airport travel if it is business-related and approved by your manager. Source: Travel Policy, Section 4.2.

11. Benefits of LangChain

Benefit	Explanation
Faster prototyping	Quickly build LLM apps without writing all glue code manually
Model flexibility	Use different LLM providers through integrations
RAG support	Connect loaders, splitters, embeddings, vector stores, and retrievers
Tool usage	Let agents call APIs, databases, calculators, or business services
Structured output	Convert LLM text into JSON-like application-friendly format
Observability support	Easier tracing and debugging with LangSmith ecosystem
Agent workflows	Build apps where LLMs reason, call tools, and complete tasks

LangChain is especially helpful when your application needs multiple GenAI components working together, not just one LLM API call.

12. Limitations of LangChain

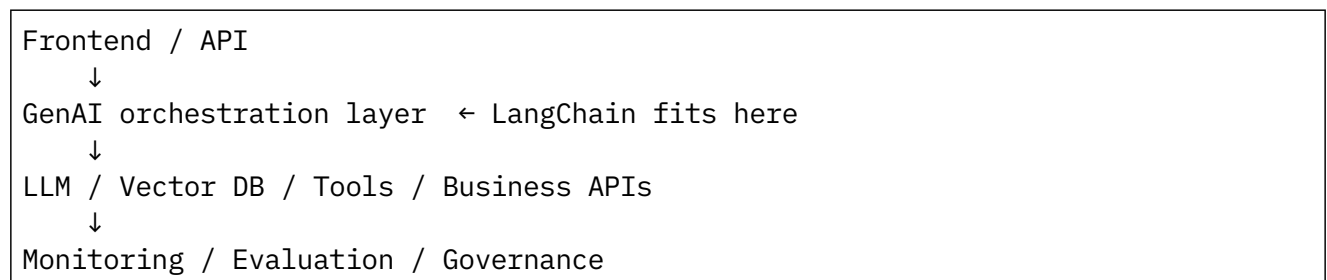
LangChain is useful, but it is not magic.

Limitation	Explanation
Version changes	LangChain has evolved quickly, so older tutorials may use outdated patterns
Hidden complexity	Simple demos are easy, but production apps still need strong engineering
Debugging difficulty	Multi-step chains and agents can be harder to debug than plain code
Cost and latency	More retrieval, tool calls, and LLM calls can increase response time and cost
Not a data-quality solution	Bad documents, poor chunking, or weak metadata still produce weak RAG
Not automatic governance	Security, access control, PII handling, and audit logs must be designed separately
Agent unpredictability	Tool-using agents need guardrails, evaluation, and monitoring

Important interview point: LangChain helps with orchestration, but production readiness comes from **architecture, evaluation, monitoring, security, and governance**.

13. Where LangChain fits in enterprise AI projects

LangChain usually fits in the **application orchestration layer**.



In an enterprise project, LangChain may be used for:

document Q&A
internal policy assistants
support chatbots
knowledge search
agentic workflows
ticket summarization
SQL question answering

```
report generation
compliance assistants
```

It should work together with:

```
FastAPI / Flask for APIs
Vector DB for retrieval
SQL / NoSQL for business data
MLflow / LangSmith for tracking
IAM / access control for security
CI/CD for deployment
Cloud platform for scaling
```

Easy examples

Example 1: Prompt template

```
Template:
"You are an IT helpdesk assistant. Answer the question: {question}"

User question:
"How do I reset my laptop password?"

Final prompt:
"You are an IT helpdesk assistant. Answer the question: How do I reset my laptop password?"
```

Example 2: Chain

```
Input question
→ format prompt
→ call LLM
→ parse answer
→ return final response
```

Example:

```
Question:
"Summarize this policy."

Chain:
Policy text + summary prompt + LLM + text parser

Output:
Short policy summary
```

Example 3: Retriever

```
Question:
"What is the notice period?"
```

Retriever searches:
Employee handbook
HR policy
Offer letter templates

Top result:
"Standard notice period is 60 days unless otherwise mentioned."

Example 4: Tool

Question:
"What is my current leave balance?"

Tool call:
get_leave_balance(employee_id="E123")

Tool result:
15 days

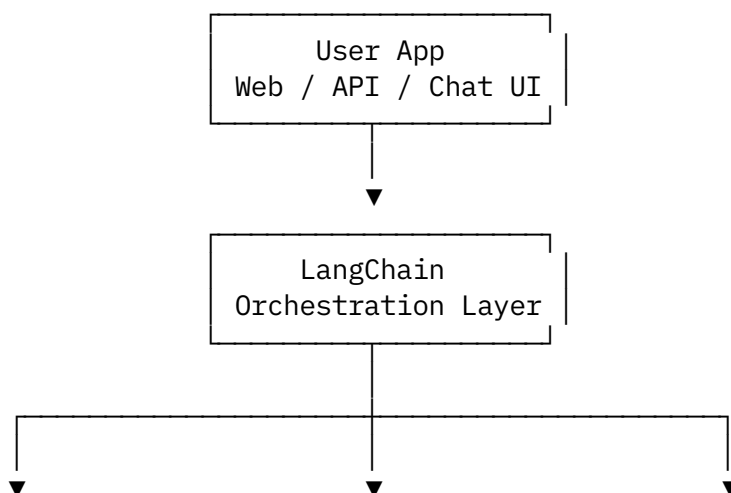
Final answer:
"You have 15 leave days available."

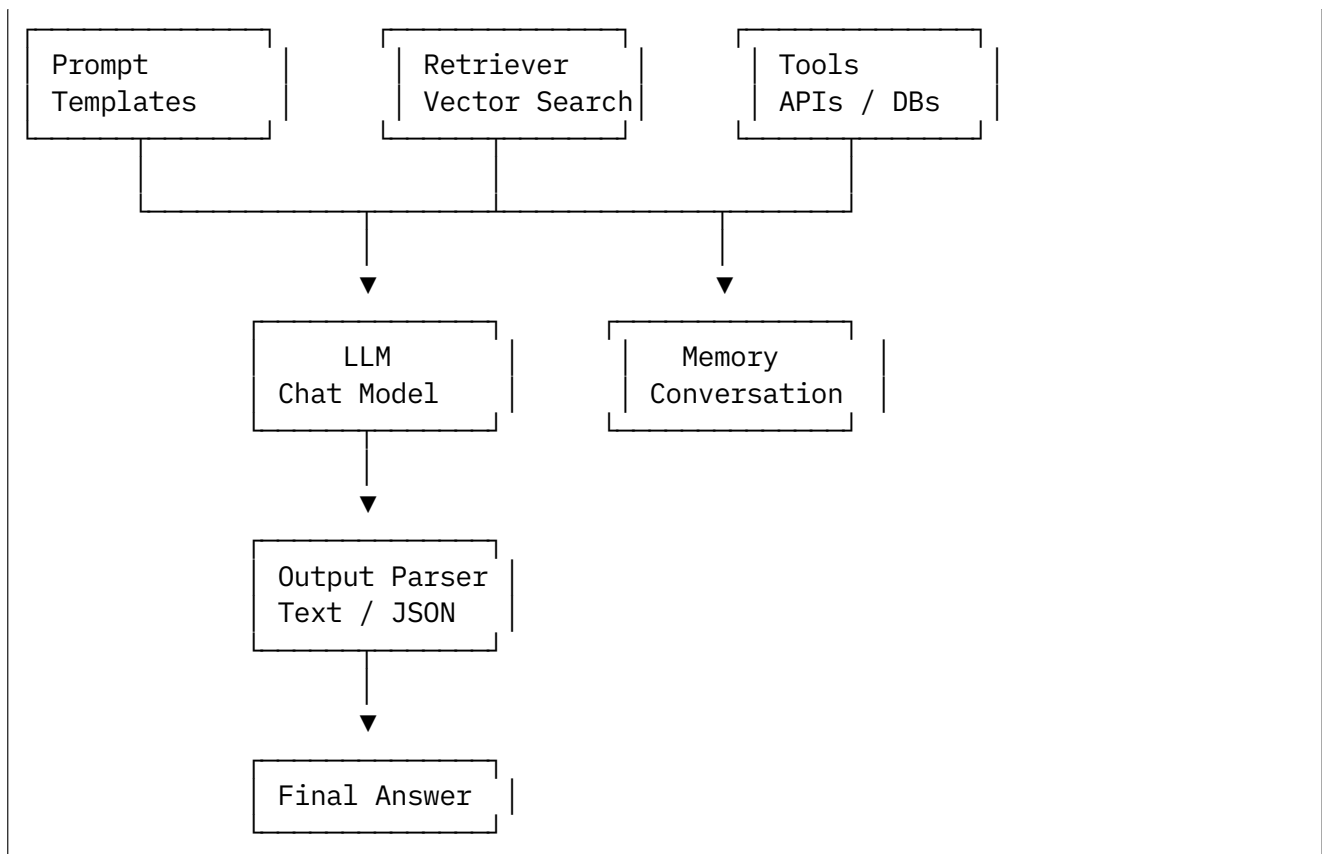
Example 5: Output parser

LLM response:
"Risk is high because the document contains missing approval."

Parsed result:
{
 "risk": "high",
 "reason": "missing approval"
}

ASCII diagram showing LangChain components





LangChain RAG app: simple flow

```
Documents
↓
Document loader
↓
Text splitter
↓
Embedding model
↓
Vector database
↓
Retriever
↓
Prompt template
↓
LLM
↓
Output parser
↓
Answer with sources
```

Pseudocode for a LangChain RAG app

```
START
1. Load enterprise documents
```

```

documents = load_files_from_folder("policy_documents/")

2. Split documents into smaller chunks
chunks = split_documents(
    documents,
    chunk_size = 500,
    overlap = 50
)

3. Convert chunks into embeddings
embeddings = embedding_model.create_embeddings(chunks)

4. Store embeddings in vector database
vector_db.store(chunks, embeddings)

5. Create retriever from vector database
retriever = vector_db.as_retriever(top_k = 3)

6. Create prompt template
prompt_template =
    "You are an internal policy assistant.
    Answer the question using only the context.
    If the answer is not present, say you do not know.

    Context:
    {context}

    Question:
    {question}

    Answer:"

7. Receive user question
question = get_user_input()

8. Retrieve relevant chunks
relevant_docs = retriever.search(question)

9. Format context
context = combine_text(relevant_docs)

10. Build final prompt
final_prompt = prompt_template.format(
    context = context,
    question = question
)

11. Send prompt to LLM
raw_answer = llm.invoke(final_prompt)

12. Parse output
final_answer = output_parser.parse(raw_answer)

13. Return answer with sources

```

```
show(final_answer)
show(source_documents)
```

END

Slightly more technical pseudocode

```
FUNCTION build_rag_app(document_path):

    documents = DocumentLoader(document_path).load()

    chunks = TextSplitter(
        chunk_size = 500,
        chunk_overlap = 50
    ).split(documents)

    vector_store = VectorStore.from_documents(
        documents = chunks,
        embedding_model = EmbeddingModel()
    )

    retriever = vector_store.as_retriever(k = 3)

    prompt = PromptTemplate(
        input_variables = ["context", "question"],
        template = """
        Use the context to answer the question.
        Do not hallucinate.
        If context is insufficient, say: I don't know.

        Context:
        {context}

        Question:
        {question}
        """
    )

    llm = ChatModel(model_name = "enterprise-approved-llm")

    parser = TextOutputParser()

    rag_chain = retriever -> prompt -> llm -> parser

    RETURN rag_chain

FUNCTION answer_question(rag_chain, user_question):

    answer = rag_chain.invoke(user_question)

    RETURN answer
```

Common mistakes

Mistake	Why it is a problem	Better approach
Thinking LangChain is an LLM	LangChain does not generate text by itself	Use LangChain to call and orchestrate LLMs
Ignoring prompt design	Weak prompts give weak answers	Use clear role, task, context, and constraints
Poor chunking	Retriever may fetch incomplete context	Tune chunk size and overlap
No metadata	Hard to filter documents by department, date, region	Store metadata like source, policy type, version
Blindly trusting RAG answers	RAG can still hallucinate	Add groundedness checks and citations
Using too many tools	Agent becomes slow and unpredictable	Start with limited, well-defined tools
Not parsing output	Raw text can break downstream apps	Use structured output or parsers
Keeping unlimited memory	Can increase cost and leak sensitive info	Use controlled memory and retention rules
Not evaluating retrieval	Bad retrieval means bad answer	Measure context precision and context recall
Depending only on LangChain	Framework is not full production architecture	Add API design, security, monitoring, CI/CD, governance

Interview-ready explanation

LangChain is best explained like this:

LangChain is an orchestration framework for LLM-powered applications. It helps connect prompts, LLMs, retrievers, tools, memory, and output parsers. In enterprise GenAI, I would use it to build RAG systems, chatbots, and agentic workflows. But I would not treat LangChain as the whole solution. For production, I would also add evaluation, monitoring, access control, logging, cost control, and deployment pipelines.

For IBM AI/GenAI roles, connect LangChain with:

RAG
Vector databases
Prompt engineering
LLM APIs
Enterprise documents
Watson / cloud services
Governance

The key idea: **LangChain helps build the GenAI workflow, but strong engineering makes it production-ready.**